Session 13 Teamwork — Give the Board Its Own Real Query, and Prove the Index That Earns It

Context

Work in your team (six teams of four students). CORE just built Issues as a complete, known-pattern resource — full CRUD, access-controlled, logged, Swagger-documented, filterable by `projectId` — with a real, proven single-field index on `projectId`. The board now needs a second, real filter: `status`. This is a requirements/proof task, not a file-by-file recipe — you decide whether the new query shape justifies a new index, and you prove your decision, the same way you'd defend it on a real team.

Goal

By the end, `GET /issues?projectId=<id>&status=<status>` works correctly, you have decided — and can defend — whether this real query shape justifies its own index beyond the one CORE already built, that decision is proven with a real `explain()` before/after (not assumed), and every existing behavior on all three resources still works exactly as before.

What to build

Your team must:

- **Add** `status` filtering to Issues' own query DTO and `findAll`, reusing the exact validated-enum pattern already used on `UpdateIssueDto`'s own `status` field.
- **Preserve** the existing `projectId` filtering — both filters must work together in one request.
- **Identify** the real query pattern this unlocks: `GET /issues?projectId=X&status=Y`, both fields together. (The endpoint will still technically accept `projectId` alone, `status` alone, or no filter at all — the query shape you are specifically optimizing today is the combined one.)
- **Measure**, with `explain("executionStats")`, the real cost of that exact compound query BEFORE adding anything new — the single-field `projectId` index from CORE already exists; does it fully serve this new query shape, or only partially?
- **Decide**, based on what you actually measured, whether a compound index is justified — and if so, **add** it with the correct field order.
- **Justify the field order in your own words** — for this exact compound query, `projectId` and `status` are both equality filters, so either order would work for THIS query alone. Why does the COMPLETE real workload (including the plain `GET /issues?projectId=X` request CORE already built) make `projectId` the right field to put first?
- **Prove** your decision with `explain()` again, using the IDENTICAL query, and compare the real numbers before and after.
- **Check**, now that the compound index exists, whether the original single-field `projectId` index from CORE is still doing anything the compound index's own prefix can't — measure the real `projectId`-only query with both indexes present, and remove the single-field index if it's now redundant (indexes aren't free; do not keep one out of habit).

- **Reconfirm** that `InternalToolGuard`/Swagger/Logger on Issues (already built in CORE) are still correct — this is a verification checklist item, not new work.
- **Preserve** all existing MongoDB CRUD/filtering behavior on Projects, Workspaces, and Issues — re-run the real curl/mongosh checks from this session's own opening as your own regression proof.

Rules

Allowed: everything already shown live — the `$type`/`@IsIn` DTO-validation pattern, `Schema.index()`, `explain("executionStats")`, Nest's own already-built `InternalToolGuard`/`Logger`/Swagger decorators. No new packages, no new decorators beyond what class already used. **Not yet:** no JWT, no users, no roles, no custom decorators, no interceptors, no Mongoose `ObjectId`/`ref`/`populate()`, no aggregation pipelines, no pagination.

Test it (design your own full proof — this is a starting point, not the whole list)

BASH

```
curl -s 'localhost:3000/issues?projectId=2&status=todo'
```

JAVASCRIPT

```
// in mongosh, against the real database — read only these three fields, both times:
db.issues.find({ projectId: 2, status: "todo" }).explain("executionStats")
// .queryPlanner.winningPlan.inputStage.indexName
// .executionStats.totalDocsExamined
// .executionStats.totalKeysExamined
```

Run this BEFORE you add anything, record the real numbers, THEN decide, THEN add your index (if justified), THEN run the identical query again and compare.

Done when

- `GET /issues?projectId=X&status=Y` returns correct results, both filters working together.
- You can state, in your own words, exactly what the baseline `explain()` measurement showed, and why it did or did not justify a new index.
- If you added a compound index: its own field order is defended with a real reason tied to this app's own actual query shape, not guessed.
- A real `explain()` before/after is recorded, with the real numbers compared, not just "it works now."
- Issues' own existing Guard/Swagger/Logger are confirmed still correct — a rejected write still returns `403` with no log line; an accepted one still logs and still shows up in `/docs`.
- `GET /projects`/`GET /workspaces` with/without filters, and every other previously-working route, still behave exactly as they did before your team's changes — proven, not assumed.
- Every team member can explain, in their own words, why `projectId` goes first in the compound index — because of the COMPLETE real workload (projectId-only requests are also real), not from a universal "always-present field first" rule.

- Your team has measured, not guessed, whether the original single-field `projectId` index is still needed now that the compound index exists, and acted on what the measurement showed.

Working time

About 50-55 minutes — most of it spent reasoning about the real query shape and measuring before deciding (twice — once before adding the compound index, once again before removing the single-field one), not typing code; the code itself is short.